6-1 什么是Form表单

需求场景

很多网站都支持上传文件，比如说：注册时上传头像；填写问卷时上传附件等等。

那么FastAPI是如何来解决文件上传的需求呢？

其实，这个需求不是FastAPI要解决的问题，这是很常见的一个需求，解决办法就是使用Form表单来上传文件。

Form表单其实是前端HTML语言中的一个标签语言，用来向服务端上传普通数据和文件。

Form表单介绍

示例1：Form表单的基本使用（浏览器直接打开该文件即可渲染出form表单页面）

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>登录页面</title>
</head>
<body>

<h1>用户登录</h1>
<form action="/login" method="post">
    <p>姓名：  <input type="text" name="username"></p>
    <p>密码：<input type="password" name="password"></p>
    <p><input type="submit" value="登录"></p>
</form>

</body>
</html>
```

- `action="/login"`，action用来设置服务端的接口，不设置表示朝当前页面所在URL发请求
- `method="post"`，methos用来设置发请求的方式，一共有两种：post和get，不设置默认是get请求
- [ ]，name用来设置该字段的字段名
- [ ]， type用来设置该字段的类型，或者说input标签的类型

补充：常见的input标签类型

```
<input type="text">          <!--普通文本-->
<input type="password">      <!--密码文本，密码密文现实-->  <!-- -->
<input type="date">          <!--日期，固定日期提交格式-->
<input type="submit">        <!--提交，用来触发form表单提交数据的动作 -->
<input type="button">        <!--按钮，普通按钮，本身没有功能，但是结合js可以绑定事件 -->
<input type="reset">         <!--重置 -->
<input type="radio">         <!--单选， -->
<input type="checkbox">      <!--多选 -->
<input type="file">          <!--文件，上传文件 -->
<input type="hidden">        <!--隐藏，隐藏当前输入框 -->
```

示例2：Form表单用来上传文件

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>上传头像</title>
</head>
<body>

<h1>上传头像</h1>
<form action="/login" method="post" enctype="multipart/form-data">
    <p>姓名： <input type="text" name="username"></p>
    <p>密码：<input type="password" name="password"></p>
    <p>头像：<input type="file" name="avatar"></p>
    <p><input type="submit" value="提交"></p>
</form>

</body>
</html>
```

- enctype="multipart/form-data"，上传文件比较特殊，需要使用enctype指定编码
- 选择文件 未选择任何文件

  ，上传文件的字段需要使用类型为file的input标签

6-2 Form表单两种编码方式

Form表单提交数据有两种请求方式，GET和POST。

当使用POST请求时，请求体数据有两种常用的编码方式，非常重要。

- ```
  <form action="/login" method="post" enctype="application/x-www-form-urlencoded"></form>
  ```
  ```

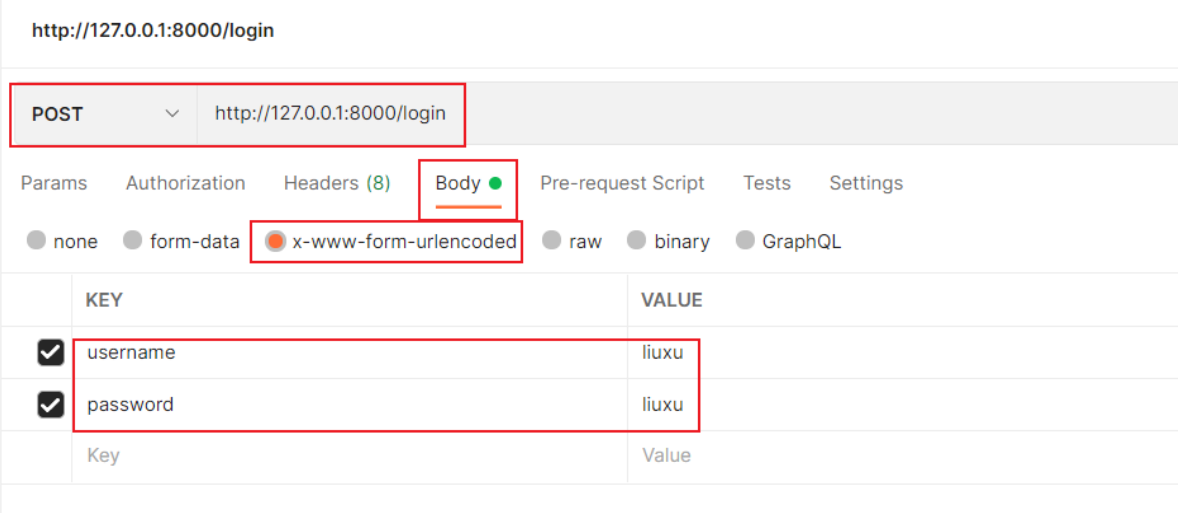
- ```
  <form action="/login" method="post" enctype="multipart/form-data"></form>
  ```

当请求体编码是application/x-www-form-urlencoded，此时编码方式和url中的查询参数编码格式一致，不能用来上传文件

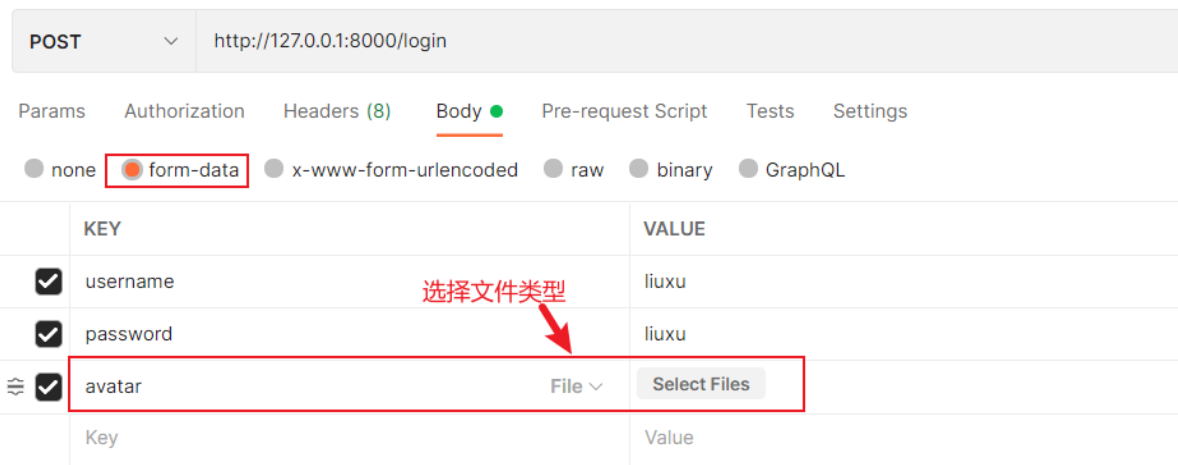
当请求体编码是multipart/form-data时，既可以上传文件，也可以用来发送普通数据。

postman中演示如何使用 Form表单上传数据

- 请求体编码是application/x-www-form-urlencoded



- 请求体编码是multipart/form-data



6-3 使用Form接收表单数据

需求场景
  ```

请求体数据的编码格式除了可以JOSN，还可以是使用form表单的application/x-www-form-urlencoded和multipart/form-data

当前端使用form表单提交数据，enctype指定application/x-www-form-urlencoded，此时FastAPI该怎么解析数据呢？

FastAPI的解决方式

示例1：使用 `Form()` 函数来接收urlencoded编码格式的表单数据

```
from fastapi import FastAPI, Form

app = FastAPI()


@app.post("/login/")
def login(username: str = Form(), password: str = Form()):
    return {
        "username": username,
        "password": password
    }
```

- 路径函数中的参数如果希望是表单字段，必须使用 `Form()`，否则会被当做查询参数和请求体参数。
- `Form()`和前面介绍的 `Path()`、`Query()`、`Body()`、`Header()`的用法基本一致，支持类型校核和转换。
- 想要使用 `Form`接收表单数据，需要安装python扩展包：`pip3 install python-multipart`

除了使用postman也可以使用FastAPI自带的api文档请求接口

6-4 使用File上传文件

【前提】：需要提前下载第三方包：`pip3 install python-multipart`

示例1：使用 `File()`用来接收表单发送过来的文件

- 路径函数中形参`file`要和表单中文件字段保持一致，且形参的类型为 `bytes`，值为`File()`，这样就可以直接将文件中二进制数据保存到形参`file`中，也就是直接将文件内容读取到内存中，适合小文件上传。

```python
from fastapi import FastAPI, File


app = FastAPI()


@app.post("/upload")
def upload_file(file: bytes = File()):
    print(file)     # file中保存的就是文件的二进制数据
    # 文件内容保存在硬盘
    with open("a.txt", "wb") as f:
        f.write(file)
    return file
```

> 使用api文档发送请求，也可以使用postman发送表单请求

示例2：形参`file`的类型不能是`str`，但如果没有类型，直接使用 `file=File()`也可以，只不过这样需要特殊处理才能获取文件内容

```python
from fastapi import FastAPI, File


app = FastAPI()


@app.post("/upload")
async def upload_file(file=File()):
    content = await file.read()     # 需要特殊处理才能过去文件的二进制数据
    print(content)
    return content
```

【重要】 对于上文件的上传需求，推荐使用示例1的方式。

【总结】只要在路径操作函数中声明了变量的类型是`bytes`且使用了`File,`则fastapi会将上传文件的内容全部去读到参数中。

6-5 使用UploadFile上传文件

【前提】：需要提前下载第三方包：`pip3 install python-multipart`

【需求】：对于大文件，不适合直接把文件内容一次性读取到内存中，此时推荐使用 `UploadFile`

示例1：使用UploadFile类型的三个属性

```
from fastapi import FastAPI, UploadFile

app = FastAPI()


@app.post("/upload")
def upload_file(file: UploadFile):      # UploadFile是类型，不用加括号哦

    content = file.file.read()          # 可以直接使用文件的操作方法
    file.file.close()

    return {
        "filename": file.filename,      # filename 是文件名 如：a.txt
        "type": file.content_type,      # content_type 是文件类型 如：text/plain
        "content": content              # file是标准的Python文件对象，可以直接使用文件
的操作
    }
```

示例2：使用UploadFile类型的四个方法

```
from fastapi import FastAPI, UploadFile

app = FastAPI()


@app.post("/upload")
async def upload_file(file: UploadFile):

    content = await file.read()     # 异步读文件
    await file.close()              # 异步关闭文件

    # await file.write()            # 异步写文件
    # await fiel.seek(1)            # 异步移动光标

    return {
        "filename": file.filename,
        "type": file.content_type,
        "content": content
    }
```

示例3：使用UploadFile的同时也可以是使用File

```python
from fastapi import FastAPI, UploadFile, File

app = FastAPI()


@app.post("/upload")
def upload_file(file: UploadFile = File()):

    return {
        "filename": file.filename,
        "type": file.content_type,
    }
```

【总结】

- UploadFile不会把文件内容全部加载到内存中，而是批量读取一定量的数据，边读边存硬盘，适合大文件。
- UploadFile类型问变量有3个属性（filename、content_type、file），4个异步方法（read\write\seek\close）。

6-6 设置上传文件是可选的

示例：设置默认值是None就表示上传文件是可选的

```python
from typing import Optional

from fastapi import FastAPI, File, UploadFile

app = FastAPI()


@app.post("/file")
def create_file(file: Optional[bytes] = File(default=None)):
    if not file:
        return {"message": "No file sent"}
    else:
        return {"file_size": len(file)}


@app.post("/uploadfile")
def create_upload_file(file: Optional[UploadFile] = None):
    if not file:
        return {"message": "No upload file sent"}
    else:
```

```
        return {"filename": file.filename}
```

6-7 上传多个文件

示例：上传多个文件，文件字段名可以同名也可以不同名

- 同名字段时，也可使用 `files: List[bytes] = File()`
- 不同名字段，可以直接定义多个形参即可。
- 多文件上传，也可以使用UploadFile；也可以同时使用File和UploadFile

```
from typing import List

from fastapi import FastAPI, File, UploadFile

app = FastAPI()


@app.post("/files")
def create_files(files: List[bytes] = File()):
    return {"file_sizes": [len(file) for file in files]}


@app.post("/files2")
def create_files(file1: bytes = File(), file2: bytes = File()):
    return {
        "file1_size": len(file1),
        "file2_size": len(file2),
    }


@app.post("/uploadfiles")
def create_upload_files(files: List[UploadFile]):
    return {"filenames": [file.filename for file in files]}


@app.post("/uploadfile2")
def upload_file_and_file(file1: UploadFile, file2: bytes = File()):
    return {
        "file1_name": file1.filename,
        "file2_size": len(file2)
    }
```

6-8 同时接收表单数据和文件

- 很多时候我们需要同时接收表单数据和上传的文件数据，此时我们可以同时使用Form和File或UploadFile

示例

```
import typing
from fastapi import FastAPI, Form, File, UploadFile

app = FastAPI()


@app.post("/register")
def register(
        avatar: typing.Optional[bytes] = File(default=None),
        username: str = Form(),
        password: str = Form(min_length=6, max_length=10),
        accessary: typing.Optional[UploadFile] = None
):
    return {
        "username": username,
        "password": password,
        "avatar": len(avatar) if avatar else 0,
        "accessary": accessary.filename if accessary else ""
    }
```

【注意】：同时使用Form和File时，File要放在Form前面。